API PAYMENT LAYER · FOR AGENTS · ON ARC

ArcPort

Pay per call. — Settled onchain. — USDC-native gas.

UNIT COST	ONCHAIN TX (PROOF RUN)	FINALITY	PRIMITIVES
$0.001 USDC / call	51 reproducible	< 1.0s deterministic	2 charge · session

WHY NANOPAYMENTS BREAK

A $0.001 call can't carry its own gas.

If every paid API call needs its own onchain payment transaction, **gas costs more than the call itself.** The unit economics turn negative, repeated agent usage becomes too expensive, and machine-to-API flow becomes operationally noisy.

PER-CALL MATH · NAIVE ONCHAIN

API call price	$0.001000
Onchain payment tx (gas)	> $0.001
Verification overhead	non-zero
Net margin to provider	**< 0**

This is the wall nanopayments hit. The fix is not a cheaper chain — it is a different payment rail for two different usage shapes.

TWO MODES · ONE RAIL

Charge for one-off. Session for repeated.

MODE · CHARGE

Charge mode

One paid API call per flow. Matches Circle Nanopayments exactly.

01	Agent requests API	HTTP
02	Gateway returns 402 challenge	OFFCHAIN
03	Agent signs X-Payment header	WALLET
04	Retry · verify · respond	ONCHAIN · SETTLE

Why it exists — simplest one-off path. Keeps $0.001 calls viable without a separate onchain payment per request.

MODE · SESSION · V2 PRIMITIVE

Session mode

Open once onchain. Sign offchain per call. Close once. Refund the unused budget.

01	Open session with fixed USDC budget	ONCHAIN · 1 TX
02	Signed calls authorized per request	OFFCHAIN · N
03	Gateway verifies usage on every call	OFFCHAIN
04	Close · refund the unused balance	ONCHAIN · 1 TX

Why it exists — amortizes onchain cost across repeated agent usage. Two onchain tx for N calls, not 2N.

ONCHAIN · SESSION PROOF RUN

Not claims.
Runbook output.

51 onchain transactions

Generated from a single reproducible session-proof run against the deployed ArcStreamChannel contract. Every tx is signed, broadcast, and explorable on Arc testnet.

17	STEP 01	USDC **approve** · session funding
17	STEP 02	Session **open** · budget locked onchain
17	STEP 03	Session **close** · refund unused budget

CONTRACT `0x594a7E570d1f915f1e11d504d0e89de28680cC98` ↗ arcscan

RUNTIME USER = THE AGENT

The web console is an **operator surface** — provision wallets, open sessions, inspect proof. The runtime user is the agent, calling the API. Three endpoints, drop-in.

AUDIENCE · 01

Agent builders

Need a payment layer for API usage. Want repeated calls priced at unit cost without paying gas per request.

uses — session mode · signed calls · N-over-2-tx

AUDIENCE · 02

Agent operators

Manage wallet identity, session budgets, and proof. Need a console to provision and inspect, not a consumer UI.

uses — web control plane · proof mode

AUDIENCE · 03

API providers

Want to monetize endpoints per call — including paid model inference — without building a payment rail from scratch.

uses — gateway · 402 · X-Payment · Gemini example

CLEAN DEMO PATH

Open. Call. Close. Prove.

01	Open Wallet · open a session	ONCHAIN · 1 TX
02	Go to Playground	CONSOLE
03	Run 3 paid Gemini Inference calls in session	OFFCHAIN · SIGNED
04	Return to Wallet · close the session	ONCHAIN · 1 TX
05	Open Proof Mode	VERIFY
06	Show open tx · close tx · spent · refund · status	DONE

TRY IT · VERIFY IT

LIVE arcport.xyz ↗

GITHUB github.com/Zhekinmaksim/arcport ↗

EXPLORER testnet.arcscan.app ↗

JUDGE DOCS docs/JUDGE_QUICKSTART.md ↗

AGENTIC ECONOMY ON ARC · HACKATHON

Pay per call. Settle onchain. Done.